

Command Pattern

Category: Behavioural Design Pattern

1. The Problem

Imagine a basic text editor with buttons for bold, italic, and underline formatting. Without the Command Pattern, each button directly interacts with the `TextEditor` class.

```
class TextEditorIn {
    public void boldText(){
        System.out.println("Text has been bolded");
    }
    public void italicizeText(){
        System.out.println("Text has been italicized");
    }
    public void underlineText(){
        System.out.println("Text has been underlined");
    }
}

class BoldButton {
    private TextEditorIn editor;
    public BoldButton(TextEditorIn editor) { this.editor = editor; }
    public void onClick(){ editor.boldText(); }
}

class ItalicButton {
    private TextEditorIn editor;
    public ItalicButton(TextEditorIn editor) { this.editor = editor; }
    public void onClick(){ editor.italicizeText(); }
}

public class WithoutCommandPattern {
    public static void main(String[] args) {
        TextEditorIn textEditorIn = new TextEditorIn();
        BoldButton boldButton = new BoldButton(textEditorIn);
        ItalicButton italicButton = new ItalicButton(textEditorIn);
        boldButton.onClick();
        italicButton.onClick();
    }
}
```

Issues: - Each button class is tightly coupled to `TextEditor` — if an action changes, every related button class needs to change too. - Hard to extend with new commands, or to add cross-cutting functionality like undo/redo or logging, since the action logic is hardcoded directly into the UI classes.

2. The Solution — Command Pattern

Intent: Encapsulate a request (an action + its receiver) as a standalone object, so that UI components (or any invoker) can trigger operations without knowing anything about how those operations are implemented.

- Buttons no longer call `TextEditor` methods directly — they hold a generic `Command` object and call `execute()` on it.
- This decouples *what triggers* an action (the invoker) from *what performs* the action (the receiver).

3. Key Idea

- **Turn a request/action into an object.** Instead of a button knowing “I call `boldText()` on the editor,” it just knows “I call `execute()` on whatever `Command` I’ve been given.”
- **Program to an interface, not an implementation** — the invoker (`Button`) only depends on the `Command` interface, never on concrete receiver classes like `TextEditor` .
- Because each command is its own object, commands can be stored, queued, logged, or reversed — enabling features like undo/redo, task scheduling, and macro recording.

4. Variants

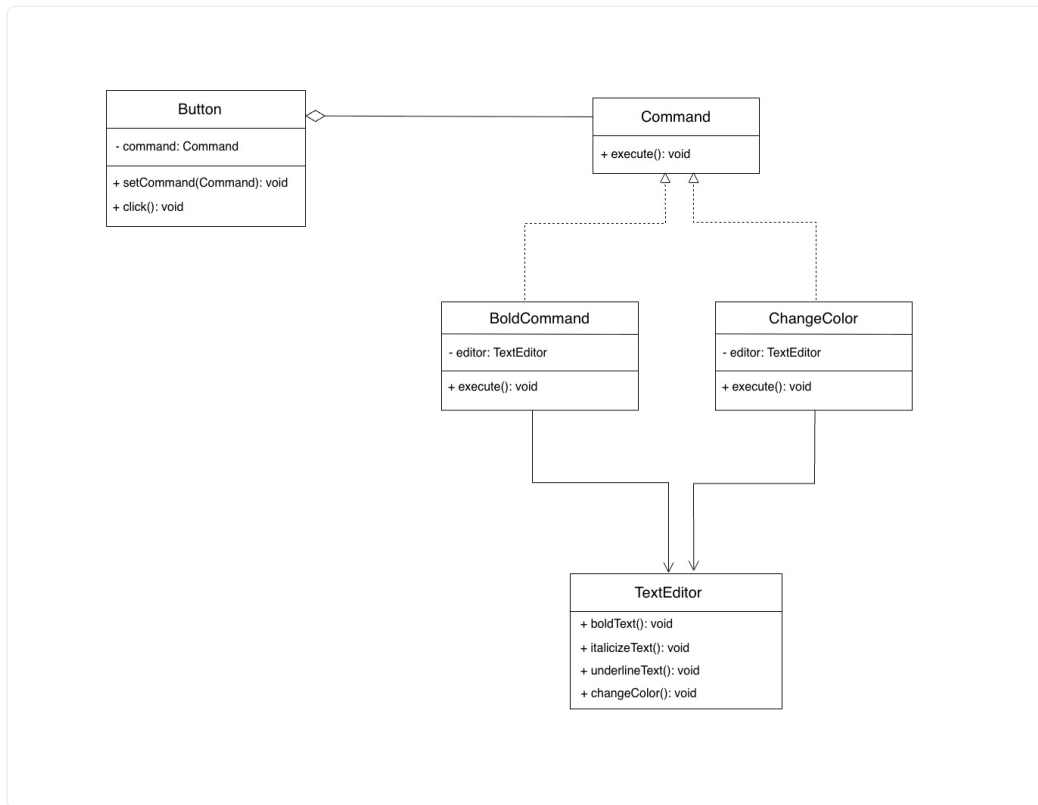
Not really applicable here — this exercise shows the standard GoF Command structure. Common extensions in practice (not shown in this code) include adding an `undo()` method to the `Command` interface for undo/redo support, or maintaining a command history/queue in the invoker.

5. Structure / Participants

Role	Responsibility	Example (from this exercise)
Command (interface)	Declares the operation to execute	<code>Command</code> → <code>execute()</code>
Concrete Command	Binds a specific action to a specific receiver	<code>BoldCommand</code> , <code>ChangeColorCommand</code>
Receiver	Knows how to actually perform the operation	<code>TextEditor</code>
Invoker	Holds a command and triggers it, without knowing what it does	<code>Button</code>

Relationship type: Composition — `Button` (invoker) *holds* a `Command` (the diamond in the diagram points from `Button` to `Command`). Each Concrete Command in turn holds a reference to its `Receiver` (`TextEditor`).

Diagram



Command Pattern UML diagram

`Button` (invoker) composes a `Command` and exposes `setCommand()` and `click()`. `Command` is the interface declaring `execute()`, implemented by concrete commands `BoldCommand` and `ChangeColor` — each of which holds a reference to the `TextEditor` (receiver) and calls the specific formatting method on it (`boldText()`, `changeColor()`, etc.) inside its own `execute()`.

6. Code — Full Implementation

```
interface Command {
    void execute();
}

class TextEditor {
    public void boldText(){
        System.out.println("Text has been bolded");
    }
    public void italicizeText(){
        System.out.println("Text has been italicized");
    }
    public void changeColor(){
        System.out.println("Color has been changed");
    }
}

// Concrete Classes for Commands
class BoldCommand implements Command {
    private TextEditor textEditor;
    public BoldCommand(TextEditor textEditor) { this.textEditor = textEditor; }
    @Override
    public void execute() { textEditor.boldText(); }
}
```

```

class ChangeColorCommand implements Command {
    private TextEditor textEditor;
    public ChangeColorCommand(TextEditor textEditor) { this.textEditor = textEditor; }
    @Override
    public void execute() { textEditor.changeColor(); }
}

class Button {
    private Command command;
    public void setCommand(Command command){ this.command = command; }
    public void click(){ command.execute(); }
}

public class CommandPattern {
    public static void main(String[] args) {
        Button button = new Button();
        TextEditor textEditor = new TextEditor();
        button.setCommand(new BoldCommand(textEditor));

        button.click();

        button.setCommand(new ChangeColorCommand(textEditor));
        button.click();
    }
}

```

What changed vs. the “without pattern” version: - Separate `BoldButton` / `ItalicButton` classes, each hardcoded to call a specific `TextEditorIn` method → a single generic `Button` class that just calls `command.execute()` . - Buttons held a direct reference to `TextEditorIn` (the receiver) → buttons now hold a `Command` (an abstraction); the receiver reference moved *into* the concrete command classes. - Adding a new button behavior used to mean writing a whole new button class → now it just means writing a new `Command` implementation; `Button` itself never changes. - Which action a button performs is now set dynamically via `setCommand()` , instead of being fixed at the button’s construction/class definition.

7. Benefits

- **Decoupling of Invoker and Receiver** — the `Button` doesn’t know anything about `TextEditor` ; it only knows the `Command` interface.
- **Command History and Undo** — commands, being objects, can be logged and reversed for undo/redo functionality.
- **Task Queuing** — commands can be stored in a queue and executed later, useful for scheduling.
- **Extensibility** — new commands (e.g. `ChangeColorCommand`) can be added without modifying any existing classes.

8. Drawbacks / Things to Watch Out For

- **Increased Complexity** — more classes are introduced, one per command, which can add up quickly.
- **Overhead** — each operation becomes a full object, which can add some memory/performance overhead in systems with a very large number of commands.

9. Real-World Use Cases

- **GUI Applications** — commands tied to buttons, menus, and keyboard shortcuts in editors, spreadsheets, drawing tools, etc.
- **Task Scheduling** — commands queued and executed later, useful for batch processing or deferred execution.
- **Undo/Redo Functionality** — commands stored and rolled back, common in IDEs, word processors, graphics software.
- **Macro Recording** — a sequence of user actions recorded as commands and replayed later as a macro.

10. Interview Questions & Answers

Q1. What is the Command Pattern? A behavioural design pattern that encapsulates a request as an object, so that the sender of a request (the invoker) is decoupled from the object that knows how to perform it (the receiver).

Q2. What problem does it solve? It removes tight coupling between UI/trigger components and the business logic they invoke, which otherwise means every trigger class must be modified whenever the underlying action changes, and makes it hard to add features like undo/redo, logging, or queuing.

Q3. What are its main components/participants? - **Command** — interface declaring the operation (`execute()`) - **Concrete Command** — binds a specific action to a specific receiver - **Receiver** — the object that actually knows how to perform the operation - **Invoker** — holds and triggers a command without knowing its details

Q4. How does the Command Pattern enable undo/redo functionality? Since each command is a standalone object (rather than an inline method call), it can store enough context to reverse itself — e.g. by adding an `undo()` method to the `Command` interface — and past commands can be kept in a history stack to step backward or forward through.

Q5. How is the Command Pattern different from the Strategy Pattern, since both wrap behavior in an object passed to a context? Structurally similar, but their intent differs: Strategy is about choosing *which algorithm* to use to accomplish a task (interchangeable “hows” for the same operation), while Command is about encapsulating *a request itself* — including its receiver and parameters — as an object, enabling things like queuing, logging, and undo that don’t naturally fit Strategy’s use case.

Q6. What are the benefits? Decoupling invoker from receiver, support for command history/undo-redo, task queuing, and easy extensibility for new commands.

Q7. What are the drawbacks? Increased number of classes (added complexity) and some memory/performance overhead from treating every operation as an object.

Q8. Can you give real-world examples? GUI button/menu/shortcut actions, task scheduling systems, undo/redo in editors and IDEs, and macro recording/playback.

Q9. What roles do the Invoker and Receiver play, and how are they connected? The Invoker (e.g. `Button`) holds a reference to a `Command` and calls `execute()` on it without knowing what happens next. The Concrete Command holds a reference to the Receiver (e.g. `TextEditor`) and, inside `execute()`, calls the actual method on the receiver that performs the real work.